

Tema 1 : Reconocimiento de Elementos en el Desarrollo de un Programa Informático

Elaborador : Xinyi Ye

Parte 1: Análisis teórico de conceptos.....	3
1. Explicación de los conceptos básicos.....	3
1.1. Describe: código fuente , código objeto y código ejecutable... 3	
1.2. Explica las fases que pasa un programa escrito en un lenguaje de programación hasta que es ejecutado en el procesador.....	3
1.3. Usa ejemplos concretos para cada concepto, mencionando en qué fase interviene cada uno en el desarrollo de un programa.....	4
2. Clasificación de lenguajes de programación.....	4
2.1. Investiga y clasifica los lenguajes de programación en función de :	5
2.1.1. Nivel de abstracción: alto, medio y bajo.....	5
2.1.2. Paradigma de programación: imperativo y declarativo....	5
(Da al menos dos ejemplos de lenguajes para cada categoría y explica brevemente por qué pertenecen a esa clasificación.)....	5
Parte 2 : Actividad práctica y de análisis.....	5
1. Identificación de paradigmas de programación a partir de ejemplos:	5
1.1. Fragmento 1 : un programa recorre una lista de números sumándolos uno por uno hasta obtener el total.(Pista: Se describe cómo se realiza la suma paso a paso.).....	5
1.2. Fragmento 2 : una consulta a una base de datos busca empleados mayores de 30 años y devuelve solo sus nombres.(Pista: Se especifica qué resultado se quiere obtener sin detallar cómo se procesa internamente.).....	6
1.3. Fragmento 3 : Un programa que calcula el factorial de un número n definiendo que el factorial de 0 es 1 y, para números mayores, multiplicando el número por el factorial del número anterior.(Pista: La lógica se define recursivamente sin especificar los pasos detallados.).....	6
2. Actividad en grupo.....	6
2.1. Seleccionen una actividad cotidiana (preparar una receta, organizar un evento, etc.) y describir la tarea de dos maneras: lenguajes imperativos y declarativas.....	6
2.2. Comparen las dos descripciones y discutan las ventajas y desventajas de cada enfoque. Incluyan esta comparación en el informe.....	7

Parte 1: Análisis teórico de conceptos

1. Explicación de los conceptos básicos

1.1. Describe: código fuente , código objeto y código ejecutable

Código fuente : es un conjunto de instrucciones escritas por un programador en un lenguaje de programación legible por humanos ,pero el ordenador todavía no puede ejecutarlo directamente.

Código objeto : es el código que resulta de traducir el código fuente a lenguaje de máquina o binario. Es un archivo intermedio y legible sólo por la computadora, pero que aún no se puede ejecutar porque le falta unirse con otras partes del sistema.

Código ejecutable : es el archivo final obtenido tras pasar el código objeto por un programa llamado “enlazador” . Contiene todas las instrucciones binarias y dependencias necesarias para que el sistema operativo pueda cargar el programa en la memoria y ponerlo a funcionar inmediatamente.

1.2. Explica las fases que pasa un programa escrito en un lenguaje de programación hasta que es ejecutado en el procesador

1. Análisis léxico , esta fase está centrada en comprobar que los elementos que se utilizan en la entrada de nuestro archivo pertenecen al lenguaje de programación que estamos utilizando.
2. Análisis sintácticos , permite determinar si los elementos que provienen del analizador léxico vienen en el orden correcto.
3. Análisis semántico , se centra en determinar si las sentencias escritas por el programador tienen sentido .
4. Generación de código intermedio , una vez que finalizan todas las fases de análisis se supone que el código del programador no tiene ningún error , por lo tanto se puede generar una representación intermedia . (Esta fase es opcional)La representación intermedia es independiente del procesador en el que se va a ejecutar el programa.
5. Optimización de código intermedio , también es una fase opcional que sirve para mejorar y “limpiar” el código generado previamente ,

haciendo que el programa funcione más rápido y consuma menos memoria.

6. Generación de código final , es la última fase donde el compilador traduce el código a código máquina o código objeto(depends del tipo de CPU que tenga el equipo)

1.3. Usa ejemplos concretos para cada concepto, mencionando en qué fase interviene cada uno en el desarrollo de un programa

1. Análisis léxico:
Naci@nalidad = español
ERROR → “Naci@nalidad” → Nacionalidad
2. Análisis sintáctico:
Tarea tener hecho
ERROR → “Tarea tener hecho” → Tener tarea hecho
3. Análisis semántico:
El coche está triste
ERROR → María está triste
4. Generación de código intermedio:
Juan compra una manzana roja y la come
 - Juan compra una manzana
 - La manzana es roja
 - Juan come la manzana
5. Optimización de código intermedio:
Juan sube para arriba y entra para adentro
 - Juan sube y entra
6. Generación de código final:
Juan sube y entra → 00011101010

2. Clasificación de lenguajes de programación

2.1. Investiga y clasifica los lenguajes de programación en función de :

2.1.1. Nivel de abstracción: alto, medio y bajo.

Nivel de abstracción alto : ocultan detalles del hardware al programador , se acerca al lenguaje natural del programador , como el inglés. Por ejemplo tenemos Java , Kotlin

Nivel de abstracción medio : mayor control sobre el hardware , pero combinado estructuras del lenguaje de alto nivel con la capacidad de manipular el sistema del bajo nivel . Nos da mayor velocidad de control y facilidad de comprender. Por ejemplo Fortran , lenguaje C

Nivel de abstracción bajo : depende de las características específicas de cada máquina , está conectado casi directamente al chip del ordenador , es muy difícil de comprender , diferentes chips puede tener diferentes instrucciones de ensamblaje , pero ejecuta muy rápido. Por ejemplo Ensamblador , Binario.

2.1.2. Paradigma de programación: imperativo y declarativo

(Da al menos dos ejemplos de lenguajes para cada categoría y explica brevemente por qué pertenecen a esa clasificación.)

Lenguajes Imperativos : crea algoritmos a través de instrucciones y experiencias(dar órdenes paso a paso a la computadora), como programación orientada a objetos y procedimental . Por ejemplo, Python y Java.

Lenguajes Declarativos : especifica valores esperados sin detallar cómo llegar al resultado , o sea decir a la computadora el resultado final que quiere , pero no cómo calcularlo paso a paso . Está programación es funcional y lógica . Por ejemplo SQL y Prolog.

Parte 2 : Actividad práctica y de análisis

1. Identificación de paradigmas de programación a partir de ejemplos:

- 1.1. Fragmento 1 : un programa recorre una lista de números sumándolos uno por uno hasta obtener el total.(Pista: Se describe cómo se realiza la suma paso a paso.)

Paradigma imperativo , porque hay que dar órdenes paso a paso al computador para obtener el resultado final , en este caso calcular el total del número de la lista .

- 1.2. Fragmento 2 : una consulta a una base de datos busca empleados mayores de 30 años y devuelve solo sus nombres.(Pista: Se especifica qué resultado se quiere obtener sin detallar cómo se procesa internamente.)

Paradigma declarativo , porque solo especifica valores esperados y no detalla cómo tiene que llegar paso a paso . Pide buscar empleados mayores de 30 años , pero no hace falta decir al computador de como hacer este movimiento de “buscar” .

- 1.3. Fragmento 3 : Un programa que calcula el factorial de un número n definiendo que el factorial de 0 es 1 y, para números mayores, multiplicando el número por el factorial del número anterior.(Pista: La lógica se define recursivamente sin especificar los pasos detallados.)

Paradigma declarativo , ya que solo espera el resultado final y no hace falta como hacer para llegar a este resultado , especifica el resultado esperado pero no detalla cómo tiene que llegar .

2. Actividad en grupo

- 2.1. Seleccionen una actividad cotidiana (preparar una receta, organizar un evento, etc.) y describir la tarea de dos maneras: lenguajes imperativos y declarativas.

Para explicar bien estos dos conceptos podemos usar el ejemplo de “comer”.

Por un lado , de manera imperativas , para obtener el objetivo final de conseguir “preparar una taza de café caliente”, los pasos son :

Paso 1 : Llena el hervidor con agua

Paso 2 : Enciende el hervidor hasta que hierva

Paso 3 : Pon una cucharada de café en la taza

Paso 4 : Agrega una cucharada de azúcar

Paso 5 : Agrega un chorro de leche

Paso 6 : Mezcla todo con una cuchara

Como resultado deseado, se obtiene una taza de café caliente , dulce y con leche.

Por otro lado , de manera declarativa , para tener una taza de café caliente , dulce y con leche , solo tienes que pedir al barista que te lo haga ,y lo tienes.

2.2. Comparen las dos descripciones y discutan las ventajas y desventajas de cada enfoque. Incluyan esta comparación en el informe.

El lenguaje imperativo describe cómo realizar operaciones paso a paso , mientras que el lenguaje declarativo indica qué resultado se desea sin detallar el procedimiento.

Ventajas del lenguaje imperativo tenemos como por ejemplo control total del procedimiento , instrucciones claras y precisas , además es fácil de replicar paso a paso y evitar el error. Al contrario el lenguaje declarativo es más corto y simple y solo se centra en el objetivo final .

Por otra parte el desventaja del lenguaje declarativo tenemos como por ejemplo ocultar detalles del proceso y que necesita un sistema que sabe que hay que hacer , mientras que el lenguaje imperativo tiene el código de texto largo y falta de flexibilidad.

PALABRAS DEL DÍA : COMPAÑEROS